STANDARDS AND INFORMATION DOCUMENTS

AES31-1-2001
r2011; s2012

AES standard for network and file transfer of audio - Audio-file transfer and exchange - Part 1: Disk format

AUDIO ENGINEERING SOCIETY, INC.
551 Fifth Avenue, New York, NY 10176, US.

STANDARDS

The AES Standards Committee is the organization responsible for the standards program of the Audio Engineering Society. It publishes technical standards, information documents and technical reports. Working groups and task groups with a fully international membership are engaged in writing standards covering fields that include topics of specific relevance to professional audio. Membership of any AES standards working group is open to all individuals who are materially and directly affected by the documents that may be issued under the scope of that working group. Complete information, including working group scopes and project status is available at http://www.aes.org/standards. Enquiries may be addressed to standards@aes.org

A standards document may be considered for "stabilized" status if it has continuing value but there is no requirement or available expertise to revise it. Any person may, at any time, propose a revision of any stabilized standard, subject to the same criteria and procedures as for new project initiations. If accepted, the project shall be assigned to the appropriate subcommittee and working group for development in the same way as for any other project. See AESSC Rules, clause 17.

The AES Standards Committee is supported in part by those listed below who, as Standards Sustainers, make significant financial contribution to its operation.

Standards Sustainer Gold

NETFLIX

Focusrite

XFRM Inc. Transforming the World One Idea at a Time

Olson Sound Design

Standards Sustainer Silver

LECTROSONICS

21ST CENTURY FOX

Audio precision

SENNHEISER

ADAMSON

THE TELOS ALLIANCE®

BOSE

Eventide

fulcrum ACOUSTIC

COVELOZ LIVE NETWORKING EXPERTS

Standards Sustainer

AUDYSSEY

NTi AUDIO

MQA

audio-technica

LAWO

CLAIR

WEISS

METRIC HALO

AEA

UNISOUND

THAT Corporation

NBCUniversal

DYNAUDIO

WorldCast Systems

Fraunhofer IIS

Control4

MERGING AUDIO FOR THE NETWORKING AGE

RAVENNA

DSPECIALISTS Digitale Audio- und Messsysteme GmbH

ARCHWAVE connecting audio

DOLBY

HARMAN A SAMSUNG COMPANY

KLIPPEL

NEUTRIK

dbx-tv

XPERI

This list is current as of 2017/7/25

AES standard for network and file transfer of audio — Audio-file transfer and exchange — Part 1: Disk format

Published by
Audio Engineering Society, Inc.

Abstract
This document defines a disk format specification to maintain compatibility with as wide a user base as possible to facilitate audio-file transfer and exchange between differing systems. The document does not describe a complete disk format but provides enough information for choosing a proprietary system that will maintain compatibility.

2011-10-17 printing

Contents

Foreword

[This foreword is not a part of *AES standard for network and file transfer of audio — Audio-file transfer and exchange — Part 1: Disk format,* AES31-1-2001.]

This document was prepared by a writing group of the SC-06-01 Working Group on Audio-File Transfer and Exchange of the SC-06 Subcommittee on Network and File Transfer of Audio, headed by K. Brown, in fulfillment of project AES-X69, Interchange Format for Digital Audio File Transport. The project was initiated in 1994.

Mark Yonge, chair
Brooks Harris, vice-chair
SC-06-01
2001-03-09

Corrigendum 2001-08-28

Formatting corrections;
ASCII definition added.

Note on normative language

NOTE In AES standards documents, sentences containing the verb "shall" are requirements for compliance with the standard. Sentences containing the verb "should" are strong suggestions (recommendations). Sentences giving permission use the verb "may." Sentences expressing a possibility use the verb "can." The decimal point is a comma except in coding, where it is a period.

AES standard for network and file transfer of audio — Audio-file transfer and exchange — Part 1: Disk format

0 Introduction

0.1 Rationale

A disk format is desired to provide a common platform so that files may be interchanged among hardware of different manufacturers of audio and video equipment.

0.2 Conventions used in this standard

0.2.1 Decimal points

According to IEC directives, the comma is used in all text to indicate the decimal point. However, in the specified coding, including the examples shown, the full stop is used as in IEC and ISO programming language standards.

0.2.2 Data representation

In this standard, all coding and data representations are printed in an equally spaced font.

0.2.3 Non-printing ASCII characters

Non-printing characters are delimited by angle brackets as in <CR> for carriage return.

0.3 Patents

The Audio Engineering Society draws attention to the fact that it is claimed that compliance with this AES standard may involve the use of the FAT file system, a proprietary scheme of Microsoft, Inc. Licensing information is available at http://209.67.75.168/hardware/fatgen.htm. It was originally developed for the IBM PC machine architecture.

The AES holds no position concerning the evidence, validity, and scope of these patent rights.

For the purposes of this document, only requirements that affect compliance with this document shall appear in the document.

1 Scope

This document defines a disk format specification to maintain compatibility with as wide a user base as possible to facilitate audio-file transfer and exchange between differing systems. The document does not describe a complete disk format but provides enough information to choose a proprietary system that will maintain compatibility.

2 Normative references

The following standards contain provisions which, through reference in this text, constitute provisions of this document. At the time of publication, the editions indicated were valid. All standards are subject to revision, and parties to agreements based on this document are encouraged to investigate the possibility of applying the most recent editions of the indicated standards.

ISO/IEC 646 (1991-01). *Information technology — ISO 7-bit coded character set for information interchange*. Geneva, CH: International Organization for Standardization.

3 Definitions and abbreviations

3.1
AES31FT
file system for use with AES31

3.2
ASCII
7-bit coded character set, per ISO 646, which describes the character encoding used by this standard

4 File allocation table (AES31FT) disk format

NOTE There are several code fragments in this document that mix data elements of different sizes, for example, 32 bit and 16 bit. It is assumed that the user has experience with software coding and understands how to properly type such operations so that data is not lost due to truncation of 32-bit values to 16-bit values. Also note that all data types are unsigned.

The file system specified shall be all little endian. One 32-bit entry shall be stored on disk as a series of four 8-bit bytes — the first being byte 0 and the last being byte 4. The positions of the 32 bits numbered 00 through 31 shall be (00 being the least significant bit):

```
byte[3]      3 3 2 2 2 2 2 2
             1 0 9 8 7 6 5 4

byte[2]      2 2 2 2 1 1 1 1
             3 2 1 0 9 8 7 6

byte[1]      1 1 1 1 1 1 0 0
             5 4 3 2 1 0 9 8

byte[0]      0 0 0 0 0 0 0 0
             7 6 5 4 3 2 1 0
```

NOTE This distinction is important for big endian machines, because translation between big and little endian will be needed to move data to and from the disk.

4.1 AES31FT regions

An AES31FT file system volume shall be composed of regions. For a 32-bit AES31FT the regions shall be laid out in the following order on the volume:

0 – reserved region;
1 – AES31FT region;
2 – file and directory data region.

4.2 Boot sector and BIOS parameter block

The first data structure of importance on an AES31FT volume shall be the BIOS parameter block (BPB), which is located in the first sector of the volume in the reserved region. This sector can often be referred to as the boot sector, the reserved sector, or even the “0th sector.” This is simply the first sector of the volume. AES31FT volumes shall have a BPB in this boot sector.

NOTE 1 In all the descriptions that follow, the fields whose names start with `BPB_` are part of the BPB. All fields whose names start with `BS_` are part of the boot sector and not really part of the BPB.

NOTE 2 Hexadecimal numbers in the coding are indicated by the prefix `0x`.

The start of sector 0 of an AES31FT volume, which contains the BPB, shall be according to table 1.

Table 1 — Boot sector and BPB structure

Name	Offset	Size (bytes)	Description
BS_JmpBoot	0x00	3	Jump instruction to boot code. This field has two allowed forms: jmpBoot[0] = 0xEB, jmpBoot[1] = 0x??, jmpBoot[2] = 0x90 and jmpBoot[0] = 0xE9, jmpBoot[1] = 0x??, jmpBoot[2] = 0x?? 0x?? indicates that any 8-bit value is allowed in that byte. This forms a three-byte unconditional branch (jump) instruction that jumps to the start of the operating system bootstrap code. This code typically occupies the rest of sector 0 of the volume following the BPB and possibly other sectors. Either of these forms is acceptable. JmpBoot[0] = 0xEB is the more frequently used format.
BS_OEMName	0x03	8	This can be MSWIN4.1. There are many misconceptions about this field since it is only a name string. For instance, certain proprietary operating systems do not pay any attention to this field. Some AES31FT drivers do. The string MSWIN4.1 is the recommended setting, because it is least likely to cause compatibility problems with proprietary systems. This field may be filled differently, but some AES31FT drivers might not recognize the volume. Typically this is an indication of what system formatted the volume.
BPB_BytsPerSec	0x0B	2	Count of bytes per sector. This value may only be 512, 1024, 2048, or 4096. For maximum compatibility the 512 value should be used. There is a lot of AES31FT code that is basically hard wired to 512 bytes per sector and does not check this field to verify it is 512. Microsoft operating systems will properly support 1024, 2048, and 4096, but these values are not recommended.
BPB_SecPerClus	0x0D	1	Number of sectors per allocation unit. This value shall be a power of 2 that is greater than 0. The legal values are 1, 2, 4, 8, 16, 32, 64, and 128. Note, however, that a value that results in a bytes per cluster value (BPB_BytsPerSec * BPB_SecPerClus) greater than 32 kbyte (32 × 1024) should never be used. Values that result in a cluster size greater than 32 kbyte will not work properly. Many programs cannot work correctly on such an AES31FT volume. For audio applications a cluster size of 32 kbyte is recommended.
BPB_RsvdSecCnt	0x0E	2	Number of reserved sectors in the reserved region of the volume starting at the first sector of the volume. This value is typically 32.
BPB_NumFATs	0x10	1	The count of AES31FT data structures on the volume. This field should always contain the value 2. Although any value greater than or equal to 1 is valid, many software programs and a few AES31FT file system drivers may not function properly if the value is something other than 2. For instance, all Microsoft file system drivers will support a value other than 2, but it is still highly recommended that no value other than 2 be used in this field. The standard value for this field is 2 to provide

			redundancy for the AES31FT data structure, so that if a sector goes bad in one of the AES31FTs, that data is not lost because it is duplicated in the other AES31FT. On non-disk-based media, such as FLASH memory cards, where such redundancy is a useless feature, a value of 1 may be used to save the space that a second copy of the AES31FT uses. However, some AES31FT file system drivers might not recognize such a volume properly.
`BPB_RootEntCnt`	`0x11`	2	Maximum root directory entries. Not applicable to this case. This field shall be set to 0.
`BPB_OldTotSec`	`0x13`	2	Number of sectors in partition smaller than 32 Mbyte. Not applicable to this case. This field shall be set to 0.
`BPB_Media`	`0x15`	1	$F8_{16}$ is the value for fixed (non-removable) media. For removable media, $F0_{16}$ is frequently used. The legal values for this field are $F0_{16}$, $F8_{16}$, $F9_{16}$, FA_{16}, FB_{16}, FC_{16}, FD_{16}, FE_{16}, and FF_{16}. The only other important point is that whatever value is put in here shall also be put in the low byte of the `FAT[0]` entry.
`BPB_OldFATSz`	`0x16`	2	Sectors per AES31FT in older AES31FT systems. Not applicable in this case. This field shall be set to 0.
`BPB_SecPerTrk`	`0x18`	2	Sectors per track for interrupt `0x13`. This field is only relevant for media that have a geometry (that is, the volume is broken down into tracks by multiple heads and cylinders) and are visible on interrupt `0x13`. This field contains the sectors per track geometry value.
`BPB_NumHeads`	`0x1A`	2	Number of heads for interrupt `0x13`. This field is relevant as discussed for `BPB_SecPerTrk`. This field contains the one-based count of heads. For example, on a 1,44-Mbyte 3,5-inch floppy drive this value is 2.
`BPB_HiddSec`	`0x1C`	4	Count of hidden sectors preceding the partition that contains this AES31FT volume. This field is generally only relevant for media visible on interrupt `0x13`. This field should always be 0 on not partitioned media. The appropriate value is operating system specific.
`BPB_TotSec32`	`0x20`	4	This field is the new 32-bit total count of sectors on the volume. This count includes the count of all sectors in all three regions of the volume. This field shall be non-zero.
`BPB_FATSz32`	`0x24`	4	This field is the 32-bit count of sectors occupied by one AES31FT.
`BPB_ExtFlags`	`0x28`	2	Bits 0–3 — Zero-based number of active AES31FT. Only valid if mirroring is disabled. Bits 4–6 — Reserved. Bit 7 — 0 means the AES31FT is mirrored at runtime into all AES31FTs. — 1 means only one AES31FT is active; it is the one referenced in bits 0–3. Bits 8–15 — Reserved.
`BPB_FSVer`	`0x2A`	2	The high byte is the major revision number. The low byte is the minor revision number. This is the version number of the AES31FT volume. This supports the ability to extend the AES31FT media type in the future without concern for old AES31FT drivers mounting the volume. This document defines the version to 0:0. If this field is non-zero, back-level operating system versions will not mount the volume. Disk utilities should respect this field and not operate on

			volumes with a higher major or minor version number than that for which they were designed. AES31FT file system drivers shall check this field and not mount the volume if it does not contain a version number that was defined at the time the driver was written.
BPB_RootClus	0x2C	4	This is set to the cluster number of the first cluster of the root directory. It may be set to 2. Disk utilities that change the location of the root directory should place the first cluster of the root directory in the first non-bad cluster on the drive (that is in cluster 2, unless it is marked bad). This is specified so that disk repair utilities can easily find the root directory if this field accidentally gets zeroed.
BPB_FSInfo	0x30	2	Sector number of FSInfo structure in the reserved area of the AES31FT volume. It may be 1. There will be a copy of the FSInfo structure in BackupBoot, but only the copy pointed to by this field updates (that is, both the primary and the backup boot records point to the same FSInfo sector).
BPB_BkBootSec	0x32	2	If non-zero, indicates the sector number in the reserved area of the volume of a copy of the boot record. It should be 6.
BPB_Reserved	0x34	12	Reserved. Code that formats AES31FT volumes should always set all of the bytes of this field to 0.
BS_DrvNum	0x40	1	Int 0x13 drive number (for example, 0x80). This field supports MS-DOS bootstrap and sets to the INT 0x13 drive number of the media (0x00 for floppy disks, 0x80 for hard disks). This field can be operating-system specific.
BS_Reserved1	0x41	1	Reserved. Code that formats AES31FT volumes should always set this byte to 0.
BS_BootSig	0x42	1	Extended boot signature (0x29). This is a signature byte that indicates that the following three fields in the boot sector are present.
BS_VolID	0x43	4	Volume serial number. This field, together with BS_VolLab, supports volume tracking on removable media. These values allow AES31FT file system drivers to detect whether the wrong disk is inserted in a removable drive. This ID can be generated by combining the current date and time into a 32-bit value.
BS_VolLab	0x47	11	Volume label. This field matches the 11-byte volume label recorded in the root directory. The AES31FT file system drivers should make sure that they update this field when the volume label file in the root directory has its name changed or created. The setting for this field when there is no volume label is the string NO NAME .
BS_FilSysType	0x52	8	Always set to the string FAT32 .

If the contents of sector 0 of an AES31FT volume are considered as a byte array, sector[510] (offset 0x01FE) equals 55_{16}, and sector[511] (offset 0x01FF) equals AA_{16}.

NOTE Many documents mistakenly say that this $AA55_{16}$ signature occupies the last two bytes of the boot sector. This statement is correct if, and only if, the bytes per sector (BPB_BytsPerSec) value is 512. If the value for BPB_BytsPerSec is greater than 512, the offsets of these signature bytes do not change. The last two bytes at the end of the boot sector may also contain this signature.

Assumptions about the value in the BPB_TotSec32 field shall be checked assuming a disk or partition of size in sectors DskSz. If the BPB_TotSec32 field is less than or equal to DskSz, the AES31FT volume is correct. It is not unusual to have a BPB_TotSec32 value that is slightly smaller than DskSz. The BPB_TotSec32 value can be considerably smaller than DskSz.

The significance is not that the AES31FT volume is damaged, but that disk space is being wasted. However, if BPB_TotSec32 is larger than DskSz, the volume can be seriously damaged or malformed because it extends past the end of the media, or overlaps data that follows it on the disk. Treating a volume for which the BPB_TotSec32 value is too large for the media or partition as valid can lead to catastrophic data loss.

4.3 AES31FT data structure

This data structure shall define a singly linked list of the extents (clusters) of a file.

> NOTE An AES31FT directory, or file container, is a regular file that has a special attribute indicating it is a directory. A directory is a file with the data or contents of the file as a series of 32-bit byte AES31FT directory entries. The AES31FT shall map the data region of the volume by cluster number. The first data cluster shall be cluster 2.

The first sector of cluster 2, the data region of the disk, shall be computed using the BPB fields for the volume. First, the count of sectors occupied by the root directory shall be determined:

```
RootDirSectors = ((BPB_RootEntCnt * 32) + (BPB_BytsPerSec – 1)) / BPB_BytsPerSec;
```

> NOTE For a 32-bit AES31FT volume the BPB_RootEntCnt value shall be 0, so the AES31FT volume RootDirSectors value will equate to 0. The 32 in the preceding is the size of one AES31FT directory entry in bytes. Note also that this computation rounds up.

The start of the data region, the first sector of cluster 2, shall be computed as follows:

```
FirstDataSector = BPB_ResvdSecCnt + (BPB_NumFATs * BPB_FATSz32) + RootDirSectors;
```

> NOTE This sector number is relative to the first sector of the volume that contains the BPB (the sector that contains the BPB is sector number 0). This does not necessarily map directly onto the drive, because sector 0 of the volume is not necessarily sector 0 of the drive due to partitioning.

Given any valid data cluster number N, the sector number of the first sector of that cluster, relative to sector 0 of the AES31FT volume, shall be computed as follows:

```
FirstSectorofCluster = ((N – 2) * BPB_SecPerClus) + FirstDataSector;
```

> NOTE Because BPB_SecPerClus is restricted to powers of 2 (1, 2, 4, 8, 16, 32, etc), division and multiplication by BPB_SecPerClus can be performed via SHIFT operations on 2's complement architectures that are usually faster instructions than MULT and DIV instructions, if the processor is not optimized for multiplication and division by powers of 2.

4.4 AES31FT type determination

4.4.1

The AES31FT type shall be determined by the count of clusters on the volume and nothing else.

> NOTE The statement is count of clusters. This count is not the same item as maximum valid cluster number, because the first data cluster is 2 and not 0 or 1.

4.4.2
The count of clusters value shall be determined by using the BPB fields for the volume as follows:

1) Determine the count of sectors occupied by the root directory.

```
RootDirSectors = ((BPB_RootEntCnt * 32) + (BPB_BytsPerSec – 1)) / BPB_BytsPerSec;
```

For a 32-bit AES31FT volume, the `BPB_RootEntCnt` value shall be 0 and therefore `RootDirSectors` will equate to 0.

2) Determine the count of sectors in the data region of the volume.

```
DataSec = BPB_TotSec32 – (BPB_ResvdSecCnt + (BPB_NumFATs * BPB_FATSz32) + RootDirSectors);
```

3) Determine the count of clusters.

```
CountofClusters = DataSec / BPB_SecPerClus;
```

NOTE This computation rounds down.

4) Determine the AES31FT type.

NOTE Off-by-one errors are possible.

4.4.3
In the following example the first number for 12-bit AES31FT is 4085; the second number for 16-bit AES31FT is 65525.

EXAMPLE

```
If(CountofClusters < 4085)
{ /* Volume is 12 bit FAT */
}else if(CountofClusters < 65525)
{/* Volume is 16 bit FAT */
}else
{ /* Volume is 32 bit FAT */
      }
```

NOTE 1 This is the only way that an AES31FT type may be determined. There is no 12-bit AES31FT volume that has more than 4084 clusters. There is no 16-bit AES31FT volume that has less than 4085 clusters or more than 65524 clusters. There is no 32-bit AES31FT volume that has less than 65525 clusters.

NOTE 2 Errors in AES31FT coding are common. When formatting an AES31FT volume that is required to have maximum compatibility with all existing AES31FT code, volumes of any type should stay at least 16 clusters away from a count of 4085 or 65625.

4.4.4
The `CountofClusters` value shall be the count of data clusters starting at cluster 2. The maximum valid cluster number for the volume shall be `CountofClusters` + 1, and the count of clusters including the two reserved clusters shall be `CountofClusters` + 2.

4.4.5
The location of the entry for a cluster of any valid cluster number N can be computed as follows:

```
FATOffset = N * 4;

ThisFATSecNum = BPB_ResvdSecCnt + (FATOffset / BPB_BytsPerSec);

ThisFATEntOffset = FATOffset % BPB_BytsPerSec;
```

NOTE The use of the modulus operator in the last of the computations in 4.4.5 will give the remainder after division of `FATOffset` by `BPB_BytsPerSec`. `ThisFATSecNum` is the sector number of the AES31FT sector that contains the entry for cluster N in the first AES31FT. If the sector number in the second AES31FT is needed, `BPB_FATSz32` is added to `ThisFATSecNum`; for the third AES31FT, 2* `BPB_FATSz32` is added, and so on.

4.4.6
Sector number `ThisFATSecNum` is a sector number relative to sector 0 of the AES31FT volume. It may be read into an 8-bit byte array named `SectorBuff`[]. Also the type `WORD` may be 16-bit unsigned and the type `DWORD` may be 32-bit unsigned. Then,

```
FATClusEntryVal = (*((DWORD *) &SectorBuff[ThisFATEntOffset])) & 0x0FFFFFFF;
```

fetches the contents of that cluster. The contents of this same cluster may be set as follows:

```
FATClusEntryVal = FATClusEntryVal & 0x0FFFFFFF;
*((DWORD *) &SectorBuff[ThisFATEntOffset]) = (*((DWORD *) &SectorBuff[ThisFATEntOffset]))
                                             &
                                             0xF0000000;

*((DWORD *) &SectorBuff[ThisFATEntOffset]) = (*((DWORD *) &SectorBuff[ThisFATEntOffset]))
                                             |
                                             FAT32ClusEntryVal;
```

In that example, an AES31FT entry is a 28-bit entry. The high 4 bits of an AES31FT entry are reserved. The high 4 bits of AES31FT entries should be changed only if the volume is formatted, at which time the whole 32-bit AES31FT entry should be zeroed, including the high 4 bits.

NOTE 1 32-bit AES31FT entries are not really 32-bit values; they are only 28-bit values. For example, all of these 32-bit cluster entry values 10000000_{16}, $F0000000_{16}$, and 00000000_{16} indicate that the cluster is free, because the high 4 bits are ignored when the cluster entry value is read. If the 32-bit free cluster value is currently 30000000_{16} and one wants to mark this cluster as bad by storing the value $0FFFFFF7_{16}$ in it, then the 32-bit entry will contain the value $x3FFFFFF7_{16}$, because the high 4 bits are preserved when the $0FFFFFF7_{16}$ bad cluster mark is written.

NOTE 2 Because the `BPB_BytsPerSec` value is always divisible by 2 and 4, an AES31FT entry never spans over a sector boundary.

4.5 Data

4.5.1 First cluster
In the directory entry, the cluster number of the first cluster of the file shall be recorded. The first cluster (extent) of the file shall be the data associated with this first cluster number, and the location of that data on the volume shall be computed from the cluster number as described in 4.3.

NOTE A zero-length file, a file that has no data allocated to it, has a first cluster number of 0 placed in its directory entry. This cluster location in the AES31FT contains either an end of clusterchain mark (EOC) or the cluster number of the next cluster of the file. See 4.4. The EOC value is AES31FT-type dependent. Assume `FATContent` is the contents of the cluster entry in the AES31FT being checked to see whether it is an EOC mark:

```
IsEOF = FALSE;

If(FATContent >= 0x0FFFFFF8)
{
    IsEOF = TRUE;
}
```

4.5.2 Last cluster

The cluster number for a cluster entry in the AES31FT that contains the EOC mark is allocated to the file and also indicates the last cluster allocated to the file. Proprietary-operating-system AES31FT-type drivers use the EOC value $0FFFFFFF_{16}$ for 32-bit AES31FT when they set the contents of a cluster to the EOC mark. However, there are various disk utilities for proprietary operating systems that use a different value.

4.5.3 `BAD_CLUSTER` mark

Any cluster that contains the `BAD_CLUSTER` value in its AES31FT entry should not be placed on the free list because it is prone to disk errors. The `BAD_CLUSTER` value shall be $0FFFFFF7_{16}$ for 32-bit AES31FT. Disk repair utilities shall recognize lost clusters that contain this special value as bad clusters and not change the content of the cluster entry.

NOTE 1 Bad clusters are also lost clusters, clusters that appear to be allocated because they contain a non-zero value but are not part of any files allocation chain.

NOTE 2 It is feasible for $0FFFFFF7_{16}$ to be an allocable cluster number on 32-bit AES31FT volumes. To avoid possible confusion by disk utilities, no AES31FT volume should be configured such that $0FFFFFF7_{16}$ is an allocable cluster number.

4.5.4 Free clusters

The list of free clusters in the AES31FT shall be a list of all clusters that contain the value 0 in their AES31FT cluster entry. This value shall be fetched for any other AES31FT entry that is not free. This list of free clusters shall not be stored on the volume; it shall be computed when the volume is mounted by scanning the AES31FT for entries that contain the value 0. For 32-bit AES31FT volumes, the `BPB_FSInfo` sector may contain a valid count of free clusters on the volume. See 4.6.

4.5.5 Reserved clusters

Two reserved clusters at the start of the AES31FT shall be utilized.

4.5.5.1

The first reserved cluster, `FAT[0]`, contains the `BPB_Media` byte value in its low 8 bits; all other bits are set to 1. For example, if the `BPB_Media` value is $F8_{16}$, `FAT[0]` contains $0FFFFFF8_{16}$.

```
ClnShutBitMask = 0x08000000;

HrdErrBitMask = 0x04000000;
```

Bit `ClnShutBitMask`	If bit is 1, volume is clean. If bit is 0, volume is dirty. Dirty indicates that the file system driver did not dismount the volume properly the last time it had the volume mounted.

4.5.5.2

The second reserved cluster, `FAT[1]`, is set by `FORMAT` to the EOC mark.

Bit `HrdErrBitMask`	If this bit is 1, no disk read/write errors were encountered. If this bit is 0, the file system driver encountered a disk I/O error on the volume the last time it was mounted, which is an indicator that some sectors may have gone bad on the volume. A disk repair utility that does surface analysis may be run to look for new bad sectors.

4.5.5.3
The AES31FT file system driver may use the high two bits of the `FAT[1]` entry for dirty volume flags (all other bits are always left set to 1).

4.5.5 Last sector
4.5.5.1
The last sector of the AES31FT may not end the AES31FT. The AES31FT stops at the cluster number in the last AES31FT sector that corresponds to the entry for cluster number `CountofClusters` + 1 (see the `CountofClusters` computation), and this entry is not necessarily at the end of the last AES31FT sector. AES31FT code should not make any assumptions about the contents of the last AES31FT sector after the `CountofClusters` + 1 entry. AES31FT format code should zero the bytes after this entry.

4.5.5.2
The `BPB_FATSz32` value may be bigger than it needs to be. In other words, there may be totally unused AES31FT sectors at the end of each AES31FT in the AES31FT region of the volume. For this reason, the last sector of the AES31FT is always computed using the `CountofClusters` + 1 value, never from the `BPB_FATSz32` value. AES31FT code should not make any assumptions about what the contents of these extra AES31FT sectors are. AES31FT format code should zero the contents of these extra AES31FT sectors.

4.6 Initialization of a 32-bit AES31FT volume

Computation of `BPB_SecPerClus` and `BPB_FATSz32`

NOTE 4.6 is specific to drives that have 512 bytes per sector. These techniques cannot be used with drives that have a different sector size. The limiting value is a volume size that is the AES31FT cut-off value. Any volume size smaller than this is a 16-bit AES31FT volume, so anything larger is a 32-bit AES31FT volume. For proprietary operating systems, this value can be 512 Mbyte. Any AES31FT volume smaller than 512 Mbyte is 16-bit AES31FT, and any AES31FT volume of 512 Mbyte or larger is 32-bit AES31FT. Many 16-bit AES31FT volumes are larger than 512 Mbyte. There are various ways to force the format to 16-bit AES31FT rather than the default of 32-bit AES31FT, and there is a great deal of code that implements different limits. This is the default cut-off value for some proprietary operating systems on volumes that have not yet been formatted.

4.6.1 Volume size code
An entry in table 2 is selected based on the size of the volume in 512-byte sectors (the value that will go in `BPB_TotSec32`), and the value that table 2 sets is the `BPB_SecPerClus` value.

Table 2 — Volume size code example

```
struct DSKSZTOSECPERCLUS
{
        DWORD DiskSize;
        BYTE SecPerClusVal;
};

/************************************************************************
   NOTE that this table includes entries for disk sizes smaller than 512 Mbyte
   even though typically only the entries for disks >= 512 Mbyte in size are
   used. The way this table is accessed is to look for the first entry in
   the table for which the disk size is less than or equal to the DiskSize
   field in that table entry. For this table to work properly BPB_RsvdSecCnt
   shall be 32, and BPB_NumFATs shall be 2. Any of these values being different
   may require the first table entries DiskSize value to be changed otherwise
   the cluster count may be to low for 32 bit AES31FT.
************************************************************************/
DSKSZTOSECPERCLUS DskTableFAT32 [] =
{
 { 66600, 0}, /* disks up to 32.5 Mbyte, the 0 value for SecPerClusVal trips an error */
 { 532480, 1}, /* disks up to 260 Mbyte, .5k cluster */
 { 16777216, 8}, /* disks up to 8 GB, 4k cluster */
 { 33554432, 16}, /* disks up to 16 GB, 8k cluster */
 { 67108864, 32}, /* disks up to 32 GB, 16k cluster */
 {0xFFFFFFFF, 64} /* disks greater than 32GB, 32k cluster */
};
```

4.6.2 `BPB_SecPerClus` value

The disk size and the AES31FT type provide a `BPB_SecPerClus` value. By computing how many sectors the AES31FT takes up, `BPB_FATSz32` can be set. `BPB_RootEntCnt`, `BPB_RsvdSecCnt`, and `BPB_NumFATs` shall be set. `DskSize` shall be the size of the volume to be put in `BPB_TotSec32`.

```
RootDirSectors = ((BPB_RootEntCnt * 32) + (BPB_BytsPerSec – 1)) / BPB_BytsPerSec;
TmpVal1 = DskSize – (BPB_ResvdSecCnt + RootDirSectors);
TmpVal2 = (256 * BPB_SecPerClus) + BPB_NumFATs;
TmpVal2 = TmpVal2 / 2;
BPB_FATSz32 = (TMPVal1 + (TmpVal2 – 1)) / TmpVal2;
```

NOTE This is how proprietary operating systems perform this computation, and it works. However, this math does not work perfectly. It will occasionally set a `BPB_FATSz32` that is up to 8 sectors too large for AES31FT, but it will never compute a `BPB_FATSz32` value that is too small.

4.7 Sector structure

4.7.1 `FSInfo` sector structure

On a 32-bit AES31FT volume, the AES31FT can be a large data structure. For this reason, a provision is made to store the last known free cluster count on the AES31FT volume. This is done so that it does not have to be computed as soon as an API call is made, asking how much free space there is on the volume (like at the end of a directory listing). The `FSInfo` sector number is the value in the `BPB_FSInfo` field; for proprietary operating systems it is set to 1. The structure of the `FSInfo` sector shall be as shown in table 3.

Table 3 — AES31FT `FSInfo` sector structure and backup boot sector

Name	Offset	Size (bytes)	Description
`FSI_LeadSig`	`0x0000`	4	Value 41615252_{16}. This lead signature is used to validate that this is in fact an `FSInfo` sector.
`FSI_Reserved1`	`0x0004`	480	AES31FT format code should always initialize all bytes of this field to 0. Currently, bytes in this field shall never be used.
`FSI_StrucSig`	`0x01E4`	4	Value 61417272_{16}. Another signature that is more localized in the sector to the location of the fields that are used.
`FSI_Free_Count`	`0x01E8`	4	Contains the last known free cluster count on the volume. If the value is $FFFFFFFF_{16}$, then the free count is unknown and shall be computed. Any other value may be used, but is not necessarily correct. It should be range checked to make sure it is less than or equal to the volume cluster count.
`FSI_Nxt_Free`	`0x01EC`	4	This is a hint for the AES31FT driver. It indicates the cluster number at which the driver should start looking for free clusters. Because an AES31FT is large, it can be time consuming if there are a lot of allocated clusters at the start of the AES31FT and the driver starts looking for a free cluster starting at cluster 2. Typically this value is set to the last cluster number that the driver allocated. If the value is $FFFFFFFF_{16}$, then there is no hint and the driver should start looking at cluster 2. Any other value can be used but it should be checked first, making sure it is a valid cluster number for the volume.
`FSI_Reserved2`	`0x01F0`	12	AES31FT format code should always initialize all bytes of this field to 0. Currently, bytes in this field shall never be used.
`FSI_TrailSig`	`0x01FC`	4	Value $AA550000_{16}$. This trail signature is used to validate that this is in fact an `FSInfo` sector. Note that the high 2 bytes of this value—which go into the bytes at offsets 510 and 511—match the signature bytes used at the same offsets in sector 0.

4.7.2 Backup sector structure

The BPB_BkBootSec field shall prevent loss of data if the contents of sector 0 of the volume are overwritten, or sector 0 goes bad and cannot be read, by reducing the severity of this problem for 32-bit AES31FT volumes, because starting at sector 6 there is a backup copy of the boot sector information including the volume's BPB code.

Only 6 should ever be placed in the BPB_BkBootSec field.

4.8 32-bit AES31FT directory structure

4.8.1 Short-file-name entry

The short-file-name (SFN) 8.3 (eight characters and an extension) directory entry shall be the basis of the 32-bit directory structure and is the foundation for the long-file-name (LFN) directory entry. See 4.8.2.

4.8.1.1 Directory structure

An AES31FT directory shall be a file composed of a linear list of 32 bytes. The only special directory, which shall always be present, is the root directory. For 32-bit AES31FT, the root directory may be of variable size and, like other directories, shall be a cluster chain. The first cluster of the root directory on an AES31FT volume shall be stored in BPB_RootClus. The root directory on any AES31FT type shall not contain any date or time stamps, shall not have a file name other than the implied file name \, and shall not contain wild-card . and .. file names as the first two directory entries in the directory. The root directory shall be the only directory on the AES31FT volume for which it is valid to have a file that has only the ATTR_VOLUME_ID attribute bit set. See table 4.

Table 4 — 32-bit AES31FT byte directory entry structure

Name	Offset	Size (bytes)	Description
Dir_Name	0x00	11	Short file name (8.3 format).
Dir_Attr	0x0B	1	File attributes: ATTR_READ_ONLY 0x01 ATTR_HIDDEN 0x02 ATTR_SYSTEM 0x04 ATTR_VOLUME_ID 0x08 ATTR_DIRECTORY 0x10 ATTR_ARCHIVE 0x20 or ATTR_LONG_NAME 0x0F The upper two bits of the attribute byte are reserved and shall always be set to 0 when a file is created and never modified or looked at after that.
Dir_NTRes	0x0C	1	Reserved. Its value shall be set to 0 when a file is created and never modified or looked at it after that.
Dir_CrtTimeTenth	0x0D	1	Milliseconds stamp at file creation time. This field actually contains a count of tenths of a second. The granularity of the seconds part of Dir_CrtTime is 2 seconds, so this field is a count of tenths of a second and its valid value range is 0–199 inclusive.
Dir_CrtTime	0x0E	2	Time file was created.
Dir_CrtDate	0x10	2	Date file was created.
Dir_LstAccDate	0x12	2	Last access date. Note that there is no last access time, only a date. This is the date of last read or write. In the case of a write, this should be set to the same date as Dir_WrtDate.
Dir_FstClusHI	0x14	2	High word of this entry's first cluster number.
Dir_WrtTime	0x16	2	Time of last write. Note that file creation is considered a write.
Dir_WrtDate	0x18	2	Date of last write. Note that file creation is considered a write.
Dir_FstClusLO	0x1A	2	Low word of this entry's first cluster number.
Dir_FileSize	0x1C	4	32-bit DWORD holding this file's size in bytes.

4.8.1.2

a) If `Dir_Name[0] == 0xE5`, then the directory entry is free (there is no file or directory name in this entry).

b) If `Dir_Name[0] == 0x00`, then the directory entry is free (same as for `0xE5`), and there are no allocated directory entries after this one (all of the `Dir_Name[0]` bytes in all of the entries after this one are also set to 0).

NOTE The special 0 value, rather than the $E5_{16}$ value, indicates to the AES31FT file system driver code that the rest of the entries in this directory do not need to be examined because they are all free.

c) If `Dir_Name[0] == 0x05`, then the actual file name character for this byte is `0xE5`. `0xE5` is actually a valid KANJI lead byte value for the character set used in Japan. The special 05_{16} value is used so that this special file name case for Japan can be handled properly and not cause the AES31FT file system code to think that the entry is free.

NOTE The `Dir_Name` field is actually broken into two, an 8-character main part and a 3-character extension. These two parts are trailing space padded with bytes of `0x20`.

d) `Dir_Name[0]` may not equal `0x20`. There is an implied '.' character between the main part of the name and the extension part of the name that is not present in `Dir_Name`. Lowercase characters are not allowed in `Dir_Name` (these characters are country specific).

The following characters shall be illegal in any bytes of `Dir_Name`:

a) values less than `0x20` except for the special case of `0x05` in `Dir_Name[0]` described in table 4;

b) `0x22`, `0x2A`, `0x2B`, `0x2C`, `0x2E`, `0x2F`, `0x3A`, `0x3B`, `0x3C`, `0x3D`, `0x3E`, `0x3F`, `0x5B`, `0x5C`, `0x5D`, and `0x7C`.

EXAMPLE

"foo.bar"	->	"FOO BAR"
"FOO.BAR"	->	"FOO BAR"
"Foo.Bar"	->	"FOO BAR"
"foo"	->	"FOO "
"foo."	->	"FOO "
"PICKLE.A"	->	"PICKLE A "
"prettybg.big"	->	"PRETTYBGBIG"
".big"	->	illegal, `Dir_Name[0]` cannot be `0x20`

In AES31FT directories all names shall be unique. Different names may refer to the same file, but there shall be one file with `Dir_Name` set, as in the example, to `FOO BAR` in any directory.

4.8.1.3

`Dir_Attr` specifies attributes of a file. The `Dir_Attr` attributes shall be as shown in table 5 according to the following procedure:

1) When a directory is created, a file with the `ATTR_DIRECTORY` bit set in its `Dir_Attr` field, its `Dir_FileSize` shall be set to 0. `Dir_FileSize` shall not be used and shall always be 0 on a file with the `ATTR_DIRECTORY` attribute (directories are sized by simply following their cluster chains to the EOC mark). One cluster shall be allocated to the directory and shall set `Dir_FstClusLO` and `Dir_FstClusHI` to that cluster number with an EOC mark placed in that cluster's entry in the AES31FT. All bytes of that cluster shall be initialized to 0. If the directory is the root directory, no further settings shall be made. If the directory is not the root directory, two special entries shall be created in the first two 32-byte directory entries of the directory (the first two 32-byte entries in the data region of the cluster just allocated).

2) The first directory entry shall have Dir_Name set to ". ".

3) The second directory entry shall have Dir_Name set to ".. ".

4) These are called the dot and dot-dot entries. The Dir_FileSize field on both entries shall be set to 0, and all of the date and time fields in both of these entries shall be set to the same values as they were in the directory entry for the directory just created. Dir_FstClusLO and Dir_FstClusHI for the dot entry (the first entry) shall be set to the same values put in those fields for the directory's directory entry (the cluster number of the cluster that contains the dot and dot-dot entries).

NOTE The dot entry is a directory that points to itself.

5) Dir_FstClusLO and Dir_FstClusHI for the dotdot entry (the second entry) shall be set to the first cluster number of the directory in which you just created the directory (the value will be 0 if this directory is the root directory).

NOTE The dot-dot entry points to the starting cluster of the parent of this directory (which is 0 if this directory's parent is the root directory).

Table 5 — Dir_Attr attributes

ATTR_READ_ONLY	Indicates that writes to the file should fail.
ATTR_HIDDEN	Indicates that normal directory listings should not show this file.
ATTR_SYSTEM	Indicates that this is an operating system file.
ATTR_VOLUME_ID	There shall be only one file on the volume that has this attribute set, and that file shall be in the root directory. This name of this file is actually the label for the volume. Dir_FstClusHI and Dir_FstClusLO shall always be 0 for the volume label (no data clusters are allocated to the volume label file).
ATTR_DIRECTORY	Indicates that this file is actually a container for other files.
ATTR_ARCHIVE	This attribute supports backup utilities. This bit is set by the AES31FT file system driver when a file is created, renamed, or written to. Backup utilities may use this attribute to indicate which files on the volume have been modified since the last time that a backup was performed.
ATTR_LONG_NAME	Indicates that the file is actually part of the long name entry for some other file. See 4.8.2 for more information.

4.8.1.4 Date and time formats

Many AES31FT file systems do not support date-time other than Dir_WrtTime and Dir_WrtDate. For this reason, Dir_CrtTimeMil, Dir_CrtTime, Dir_CrtDate, and Dir_LstAccDate are actually optional fields. Dir_WrtTime and Dir_WrtDate shall be supported, however. If the other date and time fields are not supported, they should be set to 0 on file creation and ignored on other file operations.

4.8.1.4.1 Date format

An AES31FT directory entry date stamp is a 16-bit field that is a date relative to 1980-01-01. The format (bit 0 is the LSB of the 16-bit word, bit 15 is the MSB of the 16-bit word) shall be:

Bits 0–4: Day of month, valid value range 1–31 inclusive.
Bits 5–8: Month of year, 1 = January, valid value range 1–12 inclusive.
Bits 9–15: Count of years from 1980, valid value range 0–127 inclusive (1980–2107).

4.8.1.4.2 Time format

An AES31FT directory entry time stamp is a 16-bit field that has a granularity of 2 seconds. The format (bit 0 is the LSB of the 16-bit word, bit 15 is the MSB of the 16-bit word) shall be:

Bits 0–4: 2-second count, valid value range 0–29 inclusive (0–58 seconds).
Bits 5–10: Minutes, valid value range 0–59 inclusive.
Bits 11–15: Hours, valid value range 0–23 inclusive.

The valid time range is from midnight 00:00:00 to 23:59:58.

4.8.1.5 Other considerations relating to AES31FT directories

a) Long-file-name directory entries are identical on all AES31FT types. See section 4.8.2 for more information.

b) `Dir_FileSize` is a 32-bit field. For 32-bit AES31FT volumes, the AES31FT file system driver shall not allow a cluster chain to be created that is longer than 100000000_{16} bytes, and the last byte of the last cluster in a chain that long cannot be allocated to the file. This shall be done so that no file has a file size greater than $FFFFFFFF_{16}$ bytes. This is a fundamental limit of all AES31FT file systems. The maximum allowed file size on an AES31FT volume is $FFFFFFFF_{16}$ (4 294 967 295) bytes.

c) Similarly, an AES31FT file system driver shall not allow a directory (a file that is actually a container for other files) to be larger than 65536 × 32 (2 097 152) bytes.

NOTE This limit does not apply to the number of files in the directory. This limit is on the size of the directory itself and has nothing to do with the content of the directory. There are two reasons for this limit. First, because AES31FT directories are not sorted or indexed, it is a bad idea to create huge directories; otherwise, operations like creating a new entry (which requires every allocated directory entry to be checked to verify that the name does not already exist in the directory) become very slow. Second, there are many AES31FT file system drivers and disk utilities, including proprietary ones, that expect to be able to count the entries in a directory using a 16-bit WORD variable. For this reason, directories cannot have more than 16 bits worth of entries.

4.8.2 LFN entry

LFN directory entries shall be an extension to the SFN directory entries (see 4.8.1). LFNs shall be stored in 32-byte LFN directory entries marked with attribute bytes set to $0F_{16}$ in addition to the SFN entry. For a given file or subdirectory, a group of one or more LFN directory entries shall immediately precede the single SFN directory entry on the disk; the layout of a long name directory entry appears in table 6.

Table 6 — LFN layout

Long name directory entry (n)
. . .
. . .
Long name directory entry 2
Long name directory entry 1
Existing SFN directory entry

The number of long name directory entries shall depend on the length of the long name.

Each LFN directory entry shall contain up to 13 characters of the long file name, and the operating system strings together as much as needed to comprise an entire long file name, as shown in table 7.

Table 7 — 32-bit AES31FT LFN

Name	Offset	Size (bytes)	Description
Dir_	0x00	1	Signature byte
Dir_Lname1	0x01	2	Unicode character 1
Dir_Lname2	0x03	2	Unicode character 2
Dir_Lname3	0x05	2	Unicode character 3
Dir_Lname4	0x07	2	Unicode character 4
Dir_Lname5	0x09	2	Unicode character 5
Dir_Attr	0x0B	1	File attribute: ATTR_LONG_NAME 0x0F The upper nibble of the attribute byte and shall always be 0.
Dir_Type	0x0C	1	Type indicator (shall always be 0)
Dir_Chcksm	0x0D	1	Checksum of the short 8.3 name
Dir_Lname6	0x0E	2	Unicode character 6
Dir_Lname7	0x10	2	Unicode character 7
Dir_Lname8	0x12	2	Unicode character 8
Dir_Lname9	0x14	2	Unicode character 9
Dir_Lname10	0x16	2	Unicode character 10
Dir_Lname11	0x18	2	Unicode character 11
Dir_FrstClst	0x1A	2	First cluster (shall always be 0)
Dir_Lname12	0x1C	2	Unicode character 12
Dir_Lname13	0x1E	2	Unicode character 13

The signature byte and the attribute byte shall form a validation signature that identifies the entry as a long name for the associated file. The signature byte shall contain an ordered value for each additional directory entry (each entry, from first to last, has a unique signature value).

The byte following the attribute byte, ignoring the `Dir_Type` field, shall contain a checksum or CRC of the SFN directory entry. It shall be computed by adding certain fields of the SFN directory entry and performing a modulo 256 operation on the result. The checksum shall be used to detect orphaned or corrupted LFN directory entries.

> NOTE Unicode characters are 2-byte-long characters (compared to ASCII characters where each is 1 byte long) that support traditional alphabet characters and numbers but also support other languages of the world. These include Chinese, Japanese, Korean, Arabic, Hebrew, Greek, and so on. In addition there is room for some mathematical and scientific symbols. In the 2-byte unicode characters, the first byte is always the character and the second byte is always the blank, as opposed to having the first and second bytes blank.

The first cluster field of the long name directory entry shall be restricted to zero.